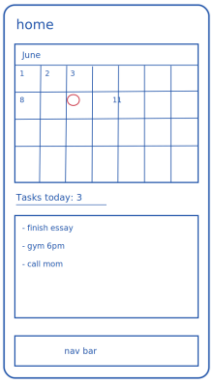
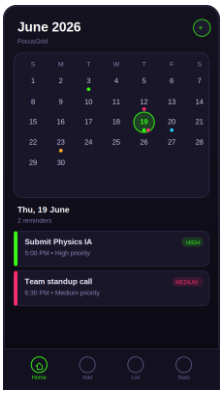
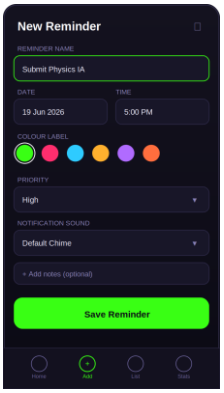
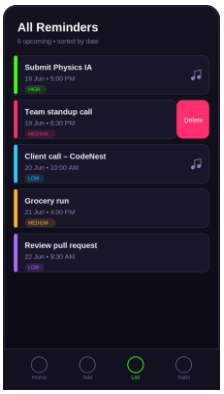
Criterion B: Design

Screens and User Feedback[1]

Initial mock-up screens for the four primary FocusGrid screens were shown to the client, Escher Lang, during the design interview (Appendix 2). Feedback was collected and incorporated into revised prototype designs. Table 2.1 documents each screen, the initial mock-up, the client feedback received, and the resulting prototype screen produced after that feedback was applied.

S/N	Initial Screen[2]	Feedback	Prototype Screen (Initial testing on VSCode)
1		1. Escher wanted colour-coded dots beneath dates that have reminders, not just a plain task counter. 2. He requested that tapping a date immediately show all reminders for that day beneath the calendar, rather than opening a separate screen.	
2		1. Escher requested a colour picker so reminders can be visually categorised by project. 2. He wanted a priority dropdown (Low / Medium / High) to help his team triage tasks. 3. An optional notes field should be collapsible to keep the form clean and uncluttered.	
3		1. The plain list was not informative enough — Escher wanted priority and time visible on each card at a glance. 2. He requested a swipe-to-delete gesture, as this felt more natural for a mobile interface than a static delete button.	

[1]The interview transcript with the client, Escher Lang, is available in Appendix 2.

[2]All initial screens were created using PowerPoint.

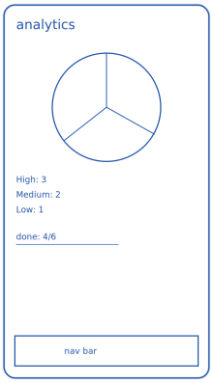
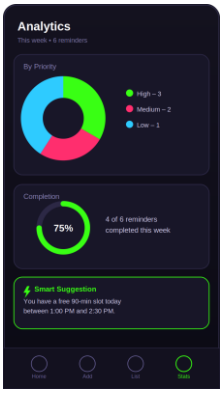
S/N	Initial Screen[2]	Feedback	Prototype Screen (Initial testing on VSCode)
4		1. A text-only summary was not visually engaging — Escher wanted a pie chart for quicker comprehension of priority distribution. 2. He also wanted a visible completion percentage showing how many reminders had been marked done versus the total for the week.	

Table 2.1: User Interface Screens and Improvements

User Interface Elements

Table 2.2 identifies and explains the purpose of the principal Flutter widgets used to construct each of the four revised screen designs shown above.

Screen	Widget	Purpose
Home (Calendar)	TableCalendar	Renders the interactive monthly calendar grid, handles date selection, and accepts a map of event markers for the colour-coded dots.
Home (Calendar)	ListView.builder	Displays the list of reminder cards for the selected date, built dynamically from the filtered Hive data.
Home (Calendar)	Container (colour dot)	Small circular containers positioned beneath calendar dates to indicate the presence and colour label of reminders on that day.
Home (Calendar)	BottomNavigationBar	Persistent navigation bar allowing the user to switch between the four main screens (Home, Add, List, Stats).
Add Reminder	TextField	Accepts text input for the reminder name and the optional, collapsible notes field.
Add Reminder	showDatePicker / showTimePicker	System dialog widgets that allow the user to select a date and time for the reminder.
Add Reminder	GestureDetector (colour swatch)	A row of six tappable circular colour containers that set the reminder's colour label.
Add Reminder	DropdownButton (priority)	Dropdown selector for reminder priority: Low, Medium, or High.
Add Reminder	DropdownButton (sound)	Dropdown selector for the notification sound associated with the reminder.
Add Reminder	ElevatedButton ('Save Reminder')	Triggers form validation and. On success, writes the reminder to Hive, schedules a notification.
Reminder List	Dismissible	Wraps each reminder card. swiping left reveal a red delete background and removes the item on confirmation.
Reminder List	Card (custom)	Displays reminder metadata - a left-edge colour bar, the title, date/time, the priority badge and a sound icon.
Reminder List	IconButton (edit)	Navigates the user to the Add Reminder screen, pre-populated with the selected reminder's existing dat
Analytics	PieChart (fl_chart)	Renders the distribution of reminders by priority as a colour-coded donut chart with a legend
Analytics	CircularProgressIndicator	Customstyled ring showing the percentage of reminders marked as completed for the current wee.
Analytics	Container (suggestion card)	Highlighted-card that displays the result of the smart free-slot detection algorithm to the user.

Table 2.2: Elements of User Interface Screens

System Flowchart

Figure 2.1 below shows the overall operating procedure of the FocusGrid application, illustrating how the bottom navigation bar links to each screen, how each screen reads from and writes to the two Hive boxes.

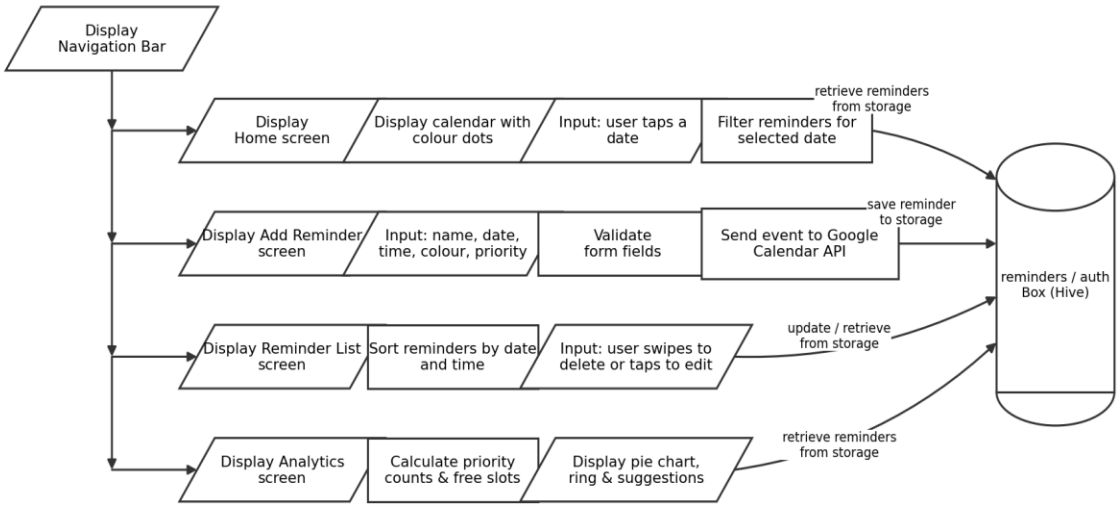


Figure 2.1: Overall System Flowchart

Individual Program Flowcharts and Pseudocode

main.dart Screen (Navigation Controller)

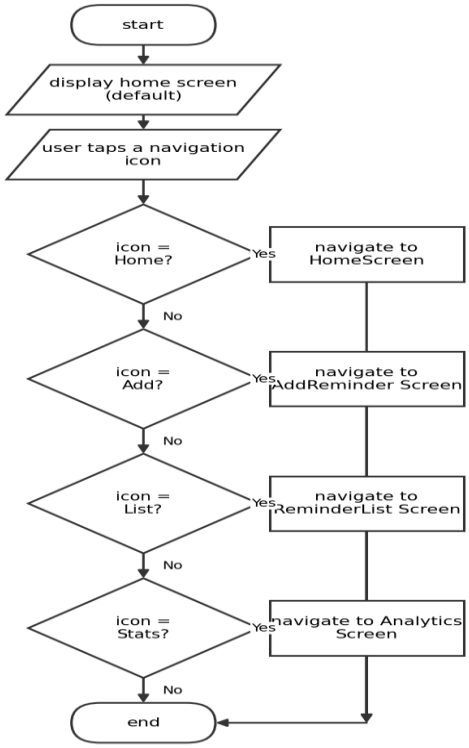


Figure 2.2: main.dart Screen Flowchart

```
Screen: MainNavigation

Display HomeScreen (default)

Navigation icon tapped:
    If icon = Home Then
        Navigate to HomeScreen
    Else If icon = Add Then
        Navigate to AddReminderScreen
    Else If icon = List Then
        Navigate to ReminderListScreen
    Else If icon = Stats Then
        Navigate to AnalyticsScreen
    End If
```

Code Snippet 2.1: main.dart Screen Pseudocode

home.dart Screen (Calendar and Daily Reminder View)

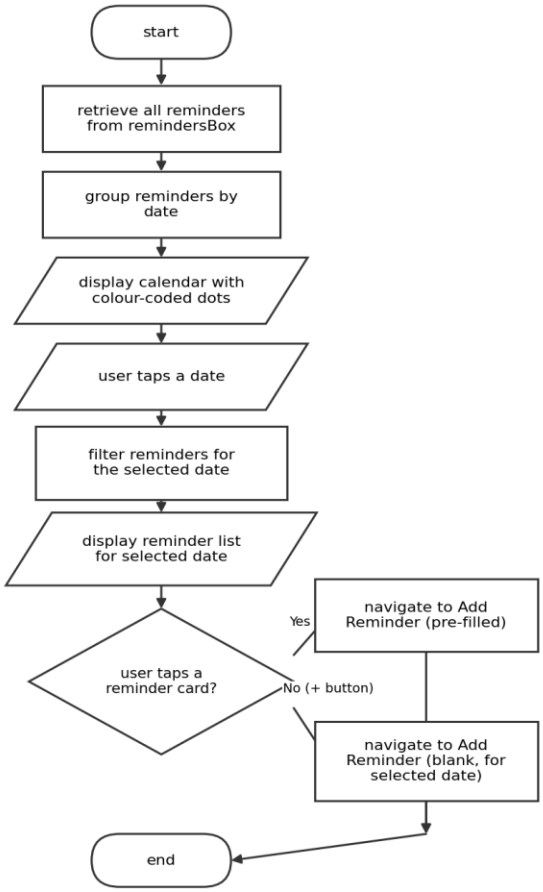


Figure 2.3: home.dart Screen Flowchart

```
Screen: HomeScreen

Load all reminders
Group reminders by date

Display calendar with colour-coded dots
Display list of reminders for the selected date

Date tapped:
    Display list of reminders for that date

Reminder card tapped:
    Navigate to AddReminderScreen with existing reminder data

"+" button pressed:
    Navigate to AddReminderScreen for the selected date
```

Code Snippet 2.2: home.dart Screen Pseudocode

addReminder.dart Screen (Save and Sync Reminder)

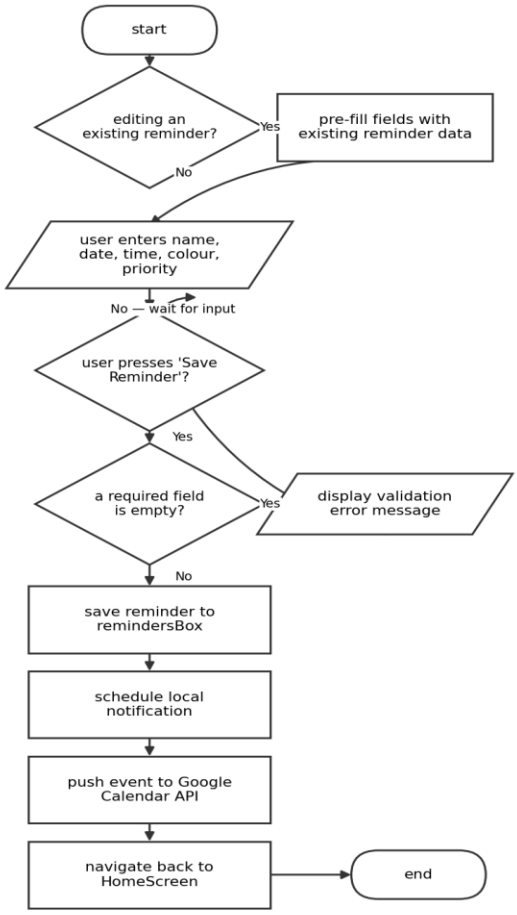


Figure 2.4: addReminder.dart Screen Flowchart

```
Screen: AddReminderScreen (existingReminder)

If existingReminder is provided Then
    Pre-fill all fields with existingReminder data
End If

Display AddReminderScreen

"Save Reminder" button pressed:
    If any required field is empty Then
        Show message "All fields are required"
    Else Then
        Save reminder details
        Schedule notification
        Navigate to HomeScreen
```

Code Snippet 2.3: addReminder.dart Screen Pseudocode

reminderList.dart Screen (All Reminders)

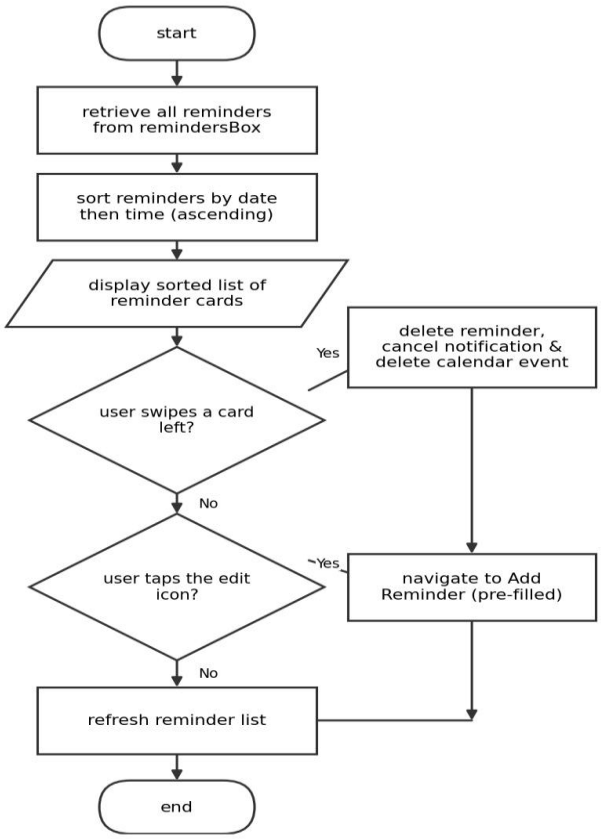


Figure 2.5: reminderList.dart Screen Flowchart

```
Screen: ReminderListScreen

Load all reminders
Sort reminders by date then time

Display sorted list of reminders

Card swiped left:
    Delete reminder
    Cancel its notification
    Refresh list

Edit icon pressed:
    Navigate to AddReminderScreen with existing reminder data
```

Code Snippet 2.5: reminderList.dart Screen Pseudocode

analytics.dart Screen (Statistics and Smart Suggestions)

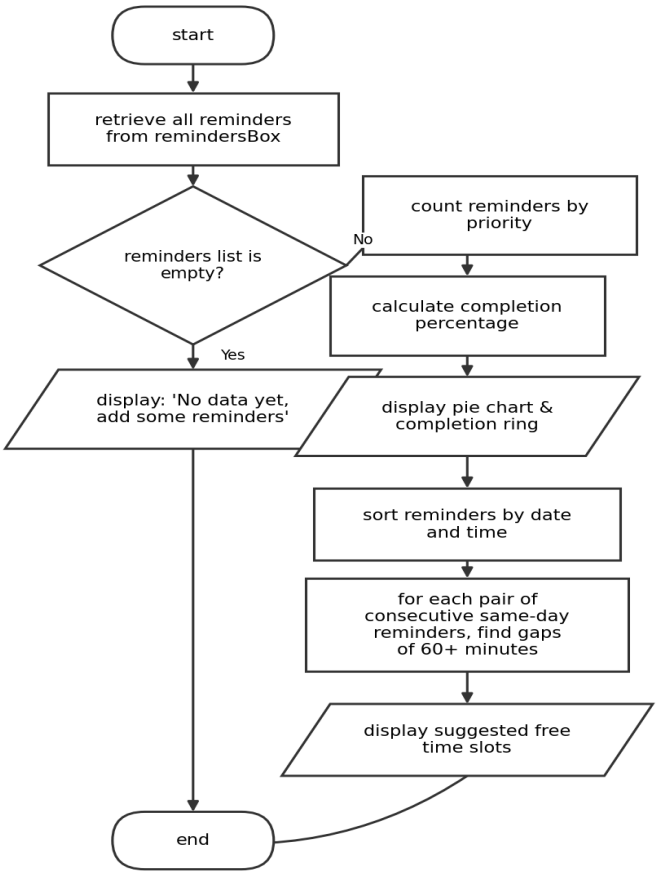


Figure 2.6: analytics.dart Screen Flowchart

```
Screen: AnalyticsScreen

Load all reminders

If no reminders exist Then
    Show message "No data yet, add some reminders"
Else Then
    Count reminders by priority
    Calculate completion percentage
    Display pie chart and completion ring

    Find free time slots:
        Sort reminders by date and time
        For each pair of reminders on the same day
            If the gap between them is 60 minutes or more Then
                Add gap to free slots list

    Display suggested free time slots
End If
```

Code Snippet 2.6: analytics.dart Screen Pseudocode

Data Structure

All the data of FocusGrid is locally stored on the device using Hive, a key-value NoSQL database for Flutter that is extremely lightweight. The data is locally stored instead of being stored in the cloud to preserve privacy of users as decided during the planning interview. There are two boxes of Hive, each having its own purpose.

1. remindersBox

Stores every Reminder object created by the user. This is the single source of truth read by HomeScreen (for the calendar dots and daily list), ReminderListScreen (for the full sorted list) and AnalyticsScreen (for priority counts and free-slot detection).

Field	Type	Description
id	int (key)	Auto-incremented primary key used to reference and cancel the notification.
date	DateTime	The selected date; used to group calendar dot markers and filter the daily list view.
time	String	Chosen time string (HH:MM); combined with date to schedule the notification and set the Calendar event start.
colourHex	String	Six-character hex colour from the colour swatch; drives the card left-edge colour and calendar dot colour.
priority	String	Low / Medium / High; used to populate the priority pie chart and sort the reminder list.
sound	String	Identifier of the notification sound selected from the dropdown.
notes	String	Optional free-text field; sent as the Calendar event description. Null when the collapsible notes section is left empty.
isDone	bool	Tracks whether the reminder has been marked complete; feeds the CircularProgressIndicator completion percentage.

Table 2.3: remindersBox Schema (Reminder Object)

2. authBox

The refreshToken allows the app to silently obtain a new accessToken once the current one expires, without further user interaction.

Field	Type	Description
refreshToken	String	Long-lived token used to silently obtain a new accessToken when the current one expires, avoiding re-authentication.
tokenExpiry	DateTime	The UTC datetime at which the current accessToken becomes invalid; checked before each API call.

Table 2.4: authBox Schema (AccessCredentials Object)

Data Flow Diagrams

Figure 2.7 (Level 0) shows the highest-level view of how data moves between the user, the FocusGrid application, and its local database storage. Figure 2.8 (Level 1) expands this further, decomposing the application process into its constituent sub-processes.

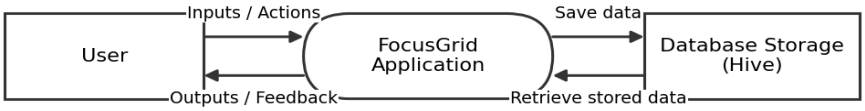


Figure 2.7: Level 0 Data Flow Diagram

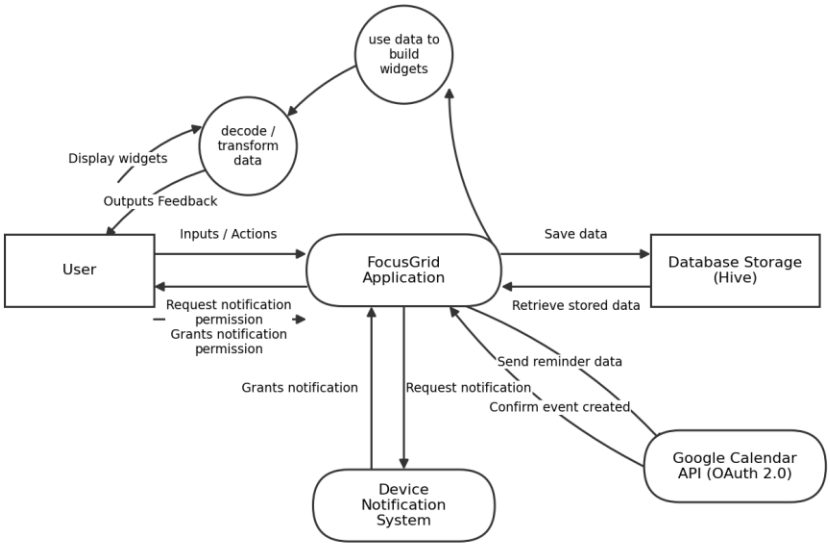


Figure 2.8: Level 1 Data Flow Diagram

Unified Modelling Language (UML) Class Diagram

Figure 2.9 illustrates the main classes comprising the FocusGrid codebase and their relations. ReminderRepository class plays the role of mediator. It is used by the UI screens and uses HiveDatabaseHelper and LocalNotificationService in order to save one reminder in such a way that it gets persisted locally, scheduled for its notification, and synced with the user's external calendar.

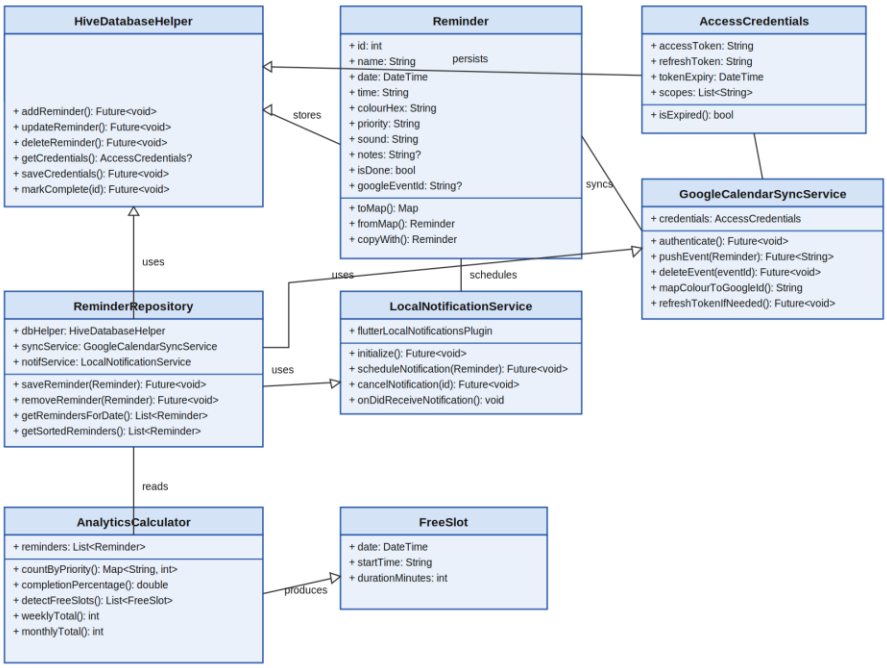


Figure 2.9: Unified Modelling Language (UML) Class Diagram

Test Plan

No.	Test	Method
main.dart Screen		
1.1	Verify that all four icons in BottomNavigationBar route to their respective screens.	Tap each icon in turn and confirm the corresponding screen is displayed without error.
home.dart Screen		
2.1	Verify that colour-coded dots appear beneath dates that have at least one reminder.	Add reminders with different colours on different dates; confirm matching dots render on the calendar.
2.2	Verify that tapping a date displays only the reminders for that specific date.	Add reminders across multiple dates; tap each date and confirm the list shown matches only that date's reminders.
2.3	Ensure that clicking on a reminder card already present results in the "Add Reminder" screen being filled up with the corresponding information.	Click on the reminder card and ensure that all fields match those of the original reminder.
addReminder.dart Screen		
3.1	Verify that the form rejects submission when the name field is left empty.	Leave the name field blank, press Save Reminder, and confirm a validation error is shown and no record is created.
3.2	Check if the colour swatch set the reminder's colour label appropriately.	Select one of the six colours, save the reminder, and check if the colourHex saved is consistent with the selected colour swatch.
3.3	Ensure that saving a reminder results in the creation of a record in Hive. Save a reminder and verify using debugPrint that the object was saved using the right field values.	Ensure that saving a reminder will set a local notification at the right date and time. Save a reminder one minute into the future and verify that you receive a system notification after that.
3.4	Ensure that creating a reminder actually creates a scheduled local notification on the right date and time.	Set a reminder one minute from now; make sure that a notification pops up from the system at that time.
reminderList.dart Screen		
4.1	Verify that all reminders are displayed sorted by date and time, soonest first.	Add reminders out of chronological order and confirm the rendered list is sorted ascending by date then time.
analytics.dart Screen		
5.1	Check whether the pie chart is reflecting the percentage of reminders in each priority group.	Create several Low/Medium/High reminders to verify that the pie chart sectors represent the expected percentages.
5.2	Check that the completion circle immediately changes its percentage when the reminder is set to done	Mark a reminder as complete and confirm the percentage shown increases without requiring an app restart.
5.3	Check that the free slot finder finds the slots which have gaps of 60 minutes or more.	Create two reminders with a gap of 90 minutes and another pair with a gap of 20 minutes and verify that the 90-minute gap is found as free.
hiveDatabaseHelper.dart File		

No.	Test	Method
6.1	Make sure that reminders remain even after the app is closed and reopened.	Add a reminder to the list, close the app using the recent apps screen on your device, and make sure that the reminder remains.
6.2	Test whether the CRUD operations in the HiveDatabaseHelper work independently.	Make use of debugPrint to print the returned result from each of the methods (add, update, delete, and getAll).

Table 2.5: Test Plan